▶ 目次

第1章　反復演算と基本プログラミング　1

1.1 Fortran 計算環境を整えよう 1
1.2 反復演算と基本プログラミング 2
1.2.1 簡単な反復演算　3
1.2.2 自由形式で見やすいプログラムを書く　9
1.2.3 次のステップに移るための準備　10
1.2.4 if 文と stop 文　11
1.2.5 無限ループ　13
1.2.6 do ループの始値・終値とストライド（増分値）　14
1.2.7 exit 文，cycle 文，goto 文　15
1.3 実数型変数を用いる反復計算 18
1.3.1 数値積分を行うプログラム　18
1.3.2 倍精度実数型変数への数値の代入　20
1.3.3 型の異なる変数を用いる演算　21
1.3.4 演算式の記述に関するその他の注意事項　22
1.4 反復計算の簡単な応用例 24
1.4.1 ニュートン法　24
1.4.2 逐次代入法　26
1.4.3 モンテカルロ法　27
1.5 入出力に関する機能 28
1.5.1 出力リダイレクション　28
1.5.2 入力リダイレクション　29
1.5.3 open 文と close 文を用いる入出力ファイル操作　30

第2章　配列を用いるプログラミング　37

2.1 配列の概要 37
2.2 配列を使う基本的な演算 38
2.2.1 ベクトルの内積を求める　38
2.2.2 配列要素の値を入力ファイルから読み取る方法　41
2.2.3 配列の上下限を超えるアクセス　42
2.2.4 配列とスカラ　43
2.3 部分配列の基本的な使い方 43
2.3.1 部分配列に対する配列代入文　43
2.3.2 配列宣言時の初期値設定　45
2.3.3 配列に対する組み込み演算　45
2.3.4 部分配列を用いる配列要素の入出力　46

2.3.5 組み込み関数 dot_product の利用　47
2.4 配列の次元と形状 48
2.4.1 多次元配列　48
2.4.2 配列の寸法，形状，大きさ　49
2.5 配列の宣言方法と割付け配列 50
2.5.1 配列を宣言する方法　50
2.5.2 割付け配列の利用例　53
2.6 多次元配列の利用例 55
2.6.1 2次元配列で行列を表現する　55
2.6.2 2次元割付け配列を利用する　56
2.6.3 多次元配列のメモリ上での配置と効率的なアクセス順序　58
2.6.4 配列に対する基本的な組み込み関数の利用　62
2.6.5 行列とベクトルの積を計算する　65
2.6.6 行列積を計算する　66
2.6.7 組み込み関数 matmul の利用　66
2.7 部分配列の利用方法 67

第3章　モジュール副プログラム　72

3.1 副プログラムに関する基本事項 72
3.1.1 複数のプログラム単位から構成されるプログラム　72
3.1.2 副プログラムの種類　73
3.2 モジュールサブルーチンの基本形 75
3.2.1 モジュールサブルーチンの記述方法　75
3.2.2 モジュールサブルーチンの利用方法　77
3.2.3 return 文　78
3.2.4 実引数と仮引数　78
3.3 局所変数とグローバル変数，仮引数の属性指定 80
3.3.1 プログラム単位ごとに独立な局所変数　80
3.3.2 save 属性を有する局所変数　80
3.3.3 仮引数に対する intent 属性　82
3.4 モジュール関数 84
3.5 配列を引数とする方法 86
3.5.1 形状明示仮配列　87
3.5.2 形状引継ぎ配列　88
3.5.3 仮配列の添字の下限について　90
3.5.4 実引数の配列と次元数が異なる仮配列の利用　92
3.5.5 配列要素を実引数として，配列の一部を仮配列として受け取る方法　95
3.5.6 未割付けの割付け配列を引数とする場合　96
3.5.7 配列の一部分を副プログラムに渡す方法　97
3.5.8 配列を引数とするモジュール関数　100
3.6 文字列を引数とするモジュール副プログラム 102

3.7 副プログラム内の自動割付け配列と局所配列 104
3.7.1 局所配列として割付け配列を利用する方法 104
3.7.2 局所配列として自動割付け配列を利用する方法 105
3.8 配列を返すモジュール関数 106
3.9 モジュールによる変数の共有 113
3.9.1 モジュールによる変数の共有と save 属性 113
3.9.2 モジュール使用宣言の位置と変数の共有 115
3.9.3 変数をモジュール内に含める記述方法 116
3.9.4 モジュール内で宣言された変数のアクセスを制限する方法 117
3.10 グローバル変数か引数か 119
3.11 モジュールの依存関係とコンパイル 120
3.11.1 複数のプログラムファイルのコンパイルの順序 120
3.11.2 モジュールの依存関係とコンパイル時の注意 123

第 4 章 外部副プログラム 125

4.1 外部副プログラムの概要 125
4.2 外部副プログラムとインターフェイス・モジュール 126
4.2.1 インターフェイス・モジュール 126
4.2.2 インターフェイス・ブロックをモジュールとする理由 127
4.2.3 インターフェイス・モジュールを用意すると利用可能となる機能 127
4.2.4 副プログラムの構成 128
4.3 外部サブルーチンの利用例 128
4.3.1 インターフェイス・モジュールと外部サブルーチンの記述例 128
4.3.2 インターフェイス・モジュールの使用宣言と only 句 131
4.3.3 インターフェイス・モジュールを用いる場合のコンパイル 132
4.4 配列を返す外部関数 133
4.5 グローバル変数モジュールの利用 134
4.6 モジュール副プログラムと外部副プログラムの比較 136
4.6.1 プログラミングに関する比較のまとめ 136
4.6.2 複数のファイルに書かれた副プログラムのコンパイル 137
4.6.3 基本的な副プログラムの構成 137

第 5 章 副プログラムの新機能 140

5.1 引数キーワードと optional 属性 140
5.2 再帰呼び出し 143
5.3 総称名 147
5.3.1 モジュール副プログラムに対する総称名 147
5.3.2 外部副プログラムに対する総称名 150

第 6 章 数値計算への応用 152

6.1 連立 1 次方程式の直接解法 152
6.1.1 連立 1 次方程式の行列表示 152
6.1.2 ガウス－ジョルダン法の計算手順 152
6.1.3 ガウス－ジョルダン法のプログラム 153
6.1.4 ガウスの消去法 155
6.1.5 ガウス－ジョルダン法とガウスの消去法の比較 157
6.1.6 部分ピボット選択 157
6.1.7 逆行列の計算 159
6.1.8 行列式の計算 160
6.1.9 最小 2 乗法による回帰多項式 160
6.2 連立 1 次方程式の反復解法 161
6.2.1 定常反復解法と反復行列 161
6.2.2 ガウス－ザイデル法のプログラム 164
6.2.3 SOR 法 165
6.2.4 非定常反復解法（Bi-CGSTAB 法） 166
6.3 ラプラス方程式の数値解法 169
6.3.1 ラプラス方程式の差分式 169
6.3.2 SOR 法によるラプラス方程式の解法 171
6.3.3 ノイマン境界条件を含むラプラス方程式の解法 174
6.4 非定常な拡散・熱伝導現象の計算 175
6.4.1 拡散方程式の陽的計算法 175
6.4.2 拡散方程式の陰的計算法 179
6.5 水面を伝わる波の計算 180
6.5.1 双曲型の微分方程式 180
6.5.2 浅水流方程式と波動方程式 181
6.5.3 特性曲線を利用する 1 次元浅水流方程式の計算法 182
6.5.4 特性曲線法による 1 次元浅水流方程式の計算プログラム 185
6.5.5 2 次元波動方程式の計算プログラム 188

第 7 章 OpenMP による並列計算 191

7.1 OpenMP の基本 191
7.1.1 進化する PC とコンパイラ 191
7.1.2 OpenMP を使ってみよう 193
7.1.3 OpenMP の基本的な記述方法 194
7.1.4 共有変数とプライベート変数，データ依存性，barrier 指示文 196
7.1.5 逐次領域の変数と並列領域のプライベート変数 198
7.2 並列領域における指示文とスレッドの制御 200
7.2.1 single 指示文 200
7.2.2 do 指示文 201
7.2.3 parallel do 指示文 203
7.2.4 reduction 節と総和を求める並列計算 204

7.2.5 critical 指示文 206
7.2.6 配列演算のデータ依存性 208
7.2.7 do 指示文と多重 do ループと演算の時間計測 208
7.2.8 行列ベクトル積 210
7.2.9 行列積の並列計算 213
7.2.10 do 指示文とスケジューリング 214
7.3 実用的な OpenMP による並列プログラミング 217
7.3.1 逐次計算プログラムへの OpenMP の導入 217
7.3.2 副プログラムへの OpenMP の導入 218
7.3.3 並列領域内からの副プログラムの呼び出し 222
7.4 OpenMP 導入時によくある誤り 224

付録 225

1 Fortran 90/95 に関する補足 225
1.1 変数の型 225
1.2 if 文の基本的な使い方 229
1.3 入出力の書式 231
1.4 書式なし入出力 235
1.5 計算誤差 236
2 組み込み手続き 240
2.1 主な組み込み関数 241
2.2 主な組み込みサブルーチン 247
2.3 組み込み手続きの利用例 247
3 グラフ描画ソフト gnuplot の簡単な使い方 259
4 コンパイルの方法に関する補足 262
4.1 make コマンドを利用する複数のファイルのコンパイル 262
4.2 make によるコンパイルの方法 265

参考文献 267
索引 268

プログラムリスト一覧

	リストの内容	ページ
1.1	反復演算により合計を計算するプログラム	3
1.2	検出しにくいタイプミスを含むプログラム例	8
1.3	偶数と奇数の和を反復計算により求めるプログラム	11
1.4	1 から入力値までの和を計算するプログラム（無限ループを使用）	13
1.5	1 から入力値までの和を計算するプログラム（exit 文，cycle 文を使用）	15
1.6	1 から入力値までの和を計算するプログラム（if 文と goto 文を使用）	16
1.7	台形公式により数値積分を行うプログラム	19
1.8	ニュートン法により $\sqrt{a}$ を求めるプログラム	26
1.9	モンテカルロ法により円周率 π を計算するプログラム例	28
1.10	入出力ファイルを利用するプログラム	30
2.1	ベクトルの内積を計算する最初のプログラム	39
2.2	定数により配列の寸法を定めるプログラム例	51
2.3	割付け配列を利用するプログラム例	52
2.4	標準入力から割付け配列の寸法を定める内積計算プログラム	53
2.5	一様乱数を 1 次元配列に格納し，変換して出力するプログラム	54
2.6	2 × 2 行列の要素の値を設定して出力するプログラム	55
2.7	割付け配列の要素の入出力を行うプログラム	56
2.8	一様乱数を要素とする上三角行列を設定するプログラム	57
2.9	2 × 2 行列を表す整数型配列の全要素を出力するプログラム	58
2.10	3 次元配列要素に 0 を代入する時間を計測するプログラム	59
2.11	四面体の 4 頂点の 3 次元座標を設定するプログラム	61
2.12	配列要素の総和，最小値および最大値などを求める例	62
2.13	配列要素をソートするプログラム例	64
2.14	行列ベクトル積を計算するプログラム（主要部分のみ）	65
2.15	行列積を計算するプログラム（主要部分のみ）	66
2.16	添字を動かして部分配列を取り出すプログラム	68
2.17	ブロック行列により行列積を計算するプログラム例	70
3.1	ファイル入力を行うための外部サブルーチンと主プログラム	73
3.2	2 つの整数型変数の値を交換するモジュールサブルーチン	75
3.3	save 属性を有する局所変数を使うプログラム例	80
3.4	円錐の体積を求めるモジュールサブルーチン	82
3.5	円錐の体積を求めるモジュール関数	85
3.6	2 次元配列に乱数を設定して出力する主プログラム	87
3.7	形状明示仮配列を使うモジュールサブルーチン	87
3.8	形状引継ぎ配列を使うモジュールサブルーチン	89
3.9	引数となる配列の添字の下限が問題となる例	90
3.10	配列の添字の下限を引数として形状引継ぎ配列を利用する例	91
3.11	大きさ引継ぎ配列を利用する例	92
3.12	形状明示仮配列により次元数の異なる配列を引数とする例	94
3.13	配列要素を実引数として，配列の一部を仮配列とする例	95

第1章 ▶ 反復演算と基本プログラミング

1.1 Fortran 計算環境を整えよう

プログラミングを習得するには，ルールや仕組みを頭で理解しただけでは不十分であり，実際にプログラムを自分の手で書いて動作を確認する，という作業を繰り返し行うことが不可欠である．このためには，読者の手元にある PC に，その作業を行えるような環境を設定しておくと便利である．PC 本体のほかに，あらかじめ何を用意すればよいかを，以下に示すプログラムの作成から実行までの手順に基づいて確認してみよう．

1. プログラムの作成：**エディタ**（テキスト文書を作成，編集するアプリケーション）を使って，Fortran 90/95 の文法に従うプログラムを書く．このプログラムを，たとえば `prog.f90` というテキストファイルとして保存する[1]．
2. コンパイル：**コンパイラ**を使用して，プログラム `prog.f90` を実行ファイルに変換する．この変換により，プログラムはテキストエディタでは読むことができない**バイナリファイル**に翻訳され，関連するファイルがリンク（結合）されて，実行ファイル（たとえば `a.out`）が作られる．実行ファイルは，シェルウィンドウなどにファイル名を入力することにより，計算機の演算処理がただちに開始される形式のファイルである．この一連の変換作業を，本書では簡単に**コンパイル**とよぶこととする．
3. 実行：`a.out` を実行し，計算結果を得る．必要であれば，得られた計算結果のグラフ表示などを行う．

上記の手順，すなわち，プログラムの作成，コンパイル，そして実行という作業を行うには，エディタとコンパイラが必要であり，そしてできればグラフ表示などを行うアプリケーションがあるとよい．このうち，テキスト文書を作成するためのエディタは，多くの PC にはすでに用意されているので問題ないであろう．PC の操作に慣れていれば，Emacs などのプログラム編集に適したエディタをインターネット上から取得して利用するとよい．UNIX PC[2] で一般に使用される Emacs 系のエディタは，Apple PC や

---

1) 計算機のデータは 2 進数（binary number）で扱われるが，そのうち文字コードなどに対応したデータはテキストエディタで文字として読むことができる．このようなデータで書かれたファイルをテキストファイルといい，可読でないデータを含むファイルをバイナリファイルという．

2) 総称的に UNIX とよばれる OS を利用する PC を，本書では UNIX PC とよぶ．SunOS，HP-UX，IRIX，FreeBSD や Linux など，UNIX にはいろいろな OS がある．macOS も UNIX をベースとしている．

Windows PC でも利用できる．

一方，コンパイラは，次のようにして PC に設定すればよい．

- インターネット上で公開されている無償のコンパイラを利用する．
- 市販のコンパイラを購入してインストールする．

Fortran 90/95 の仕様に従うものであれば，いずれも同様に動作するはずであり，どのコンパイラを利用してもよい．ただし，コンパイル時の**オプション**の付け方や，実行ファイルの演算速度などは，コンパイラにより異なる．インターネット上では，gfortran[3] など，Fortran 90/95 のコンパイラがいくつか公開されているので，これらを利用してもよいだろう．gfortran は，GNU GPL[4] というライセンスで，インターネットから入手できる．また，市販のコンパイラでも，ダウンロード後に一定の期間試用できたり，あるいは利用目的によっては無償で使用できる場合があるようなので，インターネット上で確認してみるとよい[5]．なお，複数のコンパイラを使えるようにしておくと，プログラムの問題点を発見する際に役立つことがある（まれに，コンパイラの誤動作を判別するのに役立つ場合もある）．

計算結果をグラフ表示するためのアプリケーションとしては，使い慣れたものを利用すればよいが，それが手元にない場合には，こちらもインターネット上で公開されているものを利用できる．その例として，本書では gnuplot の基本的な使い方を付録 3 で紹介している．

なお，gfortran と gnuplot のインストール方法については，本書のサポートページ

https://www.morikita.co.jp/books/mid/084722

でも紹介しているので参考にしていただきたい．

1.2　反復演算と基本プログラミング

計算機の性能を表す指標の 1 つに，GFLOPS（ギガフロップス）という単位がある[6]．1 GFLOPS は，1 秒間に 10 億回の**浮動小数点数演算**（後述する実数型変数の計算）を実行できることを意味するが，最近では個人で使用する PC やゲーム機でも GFLOPS 値は年々向上している．このように，演算を繰り返し高速に実行することは計算機が得意とするところであり，同じ演算式を数値を変えながら反復計算する方法は多くの数値

3) GNU Compiler Collection の一部として開発された GNU Fortran project の名称．詳細は https://gcc.gnu.org/wiki/GFortran を参照のこと．

4) 詳細は https://www.gnu.org/licenses/gpl-3.0.en.html を参照のこと．

5) 市販のコンパイラとしては，Intel コンパイラ（https://www.intel.com/）や PGI コンパイラ（https://www.softek.co.jp/）などがある．

6) FLOPS は FLoating point number Operations Per Second の略．G（ギガ）は国際単位系（SI）における接頭辞の 1 つで，基礎となる単位の 10 億（10^9）倍を意味する．

計算の基本となっている．本章では，簡単な反復演算を中心に，関連するプログラミングの基本事項を確認していく．

1.2.1　簡単な反復演算

■プログラムを作成する前に　前節の記述などを参考にして，読者の PC には Fortran 90/95 の計算を行う準備が整っているであろう．プログラムは，ホームディレクトリ上にいきなり作成するのではなく，計算対象やテーマに応じた**ディレクトリ**[7] を作り，その中に関連するデータなどとともに納めておくのがよい．ここでは，たとえば keisan というディレクトリを作成する．そして，その中に loop というディレクトリを作り，この loop ディレクトリ内に以下に示すプログラムを作成することとしよう．このように，作成するプログラムをディレクトリごとに整理しておくと，後々の取り扱いが容易になる．

■合計を計算するプログラム　最初に，反復演算により 1 から 100 までの和を求めて結果を出力する，というリスト 1.1 に示されたプログラムを作成しよう．内容の詳細は後で確認することとして，ここではまずリスト 1.1 のプログラムをエディタで作成して，コンパイルした後，実行してみよう．なお，記号 ! とそれより右側にある説明文を打ち込む必要はない．また，プログラムを見やすくするため，各行の先頭を**字下げ**（インデント）して記述することに注意しよう[8]．字下げの詳細については 1.2.2 項で述べる．

リスト 1.1　反復演算により合計を計算するプログラム

```
program loop                ! プログラム文 (プログラムの名称を指定)
  implicit none             ! 暗黙の変数型の宣言を無効化する
  integer i, wa             ! 整数型変数 i と wa を宣言
  wa = 0                    ! 合計を求める変数 wa をゼロクリア
  do i = 1, 100             ! do ループによる反復演算
     wa = wa + i
  enddo
  write(*, *) 'wa = ', wa   ! 結果の出力
end program loop            ! end 文
```

リスト 1.1 のプログラムを，たとえば loop.f90 という名前のファイルとして保存する．ファイル名のドット以降の部分，すなわち .f90 は**拡張子**といわれる．Fortran 90/95 のプログラムのファイル名の拡張子は，.f ではなく，通常 .f90 とする．このようにすると，エディタやコンパイラに Fortran 90/95 のプログラムとして解釈され，都合がよい．

7) 以降，Apple PC や Windows PC では「ディレクトリ」は「フォルダ」と適宜読み替える．

8) Emacs 系のエディタでは，Tab（タブ）キーを行内で押すと，自動的に字下げができる．

このファイルを gfortran や市販のコンパイラを使用してコンパイルし，実行する．gfortran を用いる場合のプログラムのコンパイルと実行方法については，サポートページを参照されたい．これらが正しく行われると，次の結果がディスプレイ上に表示される．

```
wa =  5050
```

このようにプログラムの動作が確認できたところで，以下ではその内容を詳しく見ていくことにしよう．

■プログラムの構成　リスト 1.1 に示されたプログラムの全体的な構成は，次のようになっている．

主プログラムの記述形式

```
program プログラムの名称
  宣言文 (implicit none 宣言，変数の宣言)
  実行文 (総和の計算と結果の出力)
end program プログラムの名称
```

リスト 1.1 では，1 行目がプログラムの名称を記述する**プログラム文**，2，3 行目が**宣言文**である．これらの宣言文では，implicit none 宣言とプログラム中で使用する変数の宣言が行われている．宣言文の後に，プログラムの本体である，合計を計算するための**実行文**が続く．このように，宣言文は実行文よりも上に書く規則となっている．そして，プログラムの最後には，end 文を記述する．

上記のように，program 文から始まり，end 文で終わる**プログラム単位**（program unit）を**主プログラム**（main program）という．プログラム単位の詳細は，3.1 節で解説する．一般のプログラムでは，主プログラムと第 3 章以降で扱う副プログラムが組み合わされた形となるが，第 2 章までは，主プログラムのみから構成されるプログラムを考えることとする．

■プログラムと変数の名称　リスト 1.1 の第 1 行目にある program 文には，次のように適当な**プログラムの名称**を記述する．この program 文は省略可能だが，プログラムを読みやすくするため，本書では省略しないこととする[9]．

```
program プログラムの名称
```

リスト 1.1 の例では，loop というのがこのプログラムの名称である．プログラムの名称は，先頭の文字が英字であることが必要で，2 番目以降の文字には英数字とアンダースコア _ が使用できる．名称の長さは 31 字まで許されるが[10]，プログラム名は内容を表す簡潔なものがよい．プログラムの名称に関するこのような規則は，プログラム中で用いる変数（リスト 1.1 の例では i と wa）にも同じように適用される．

9) 「program プログラムの名称」を省略した場合には，end 文に続く「program プログラムの名称」も省略する．end 文そのものは省略できない．

10) Fortran 95 対応のコンパイラでは，さらに長い名称が許される．

プログラム名や**変数名**に関して，初心者には次のような間違いがよく見られる．

- プログラム名や変数名の先頭に，数字などの英字以外の文字を使う．
- プログラム名や変数名に，ハイフンやピリオドなど，許可されていない文字を入れてしまう．
- プログラムや変数名，またプログラム文中に日本語などの**2 バイト文字（全角文字）**を使う[11]．
- プログラム名と同じ変数名を使う．
- Fortran で用意されている**組み込み手続き**（付録 2 参照）の名称と同じ名前のプログラムとしたり，組み込み手続きと同じ名前の変数を宣言する．そして，そのプログラム中で，同一名称の組み込み手続きを使用する．たとえば，プログラム名称を program sin として，その中で組み込み関数 sin を使用する[12]．

上記の誤りがあるとコンパイル時にエラーが出るので，プログラムを修正して再びコンパイルする．リスト 1.1 のプログラムを正しく実行できた読者は，上記の間違いを含んだプログラムを作成して，コンパイル時にどのようなエラー表示が出るかを見てみるとよい．

主プログラムの最終行には，リスト 1.1 に示されるように end 文を書く．end 文を省略することは許されず，またプログラムの最終行以外のところに書くことはできない．リスト 1.1 では，end 文の後にプログラム名称を付けている．この名称は省略可能であるが，明示したほうがわかりやすいプログラムとなる[13]．

■コメント領域と宣言文　リスト 1.1 では，記号！とその右側に各行の説明が書かれている．記号！と，それより右側にある同一行の部分は，**コメント領域**，あるいは**注釈**といわれ，プログラム部分とはみなされない．このため，コメント領域には，プログラムの説明などを自由に記述することができる．リスト 1.1 のように，日本語（2 バイト文字）を使用してもよい．また，行頭に！を付ければその行が無視されるので，実行文の動作を一時的に停止させることができる．これを利用して，プログラムの動作確認やエラーの検出を行ってもよい．

11) とくに，全角の空白を入れてしまうと，問題の箇所は「空白」なので，プログラムを見ただけではエラーの原因がわからないため注意しよう．なお，コメント領域（記号！以降の同一行の部分）と文字列を扱う部分（アポストロフィ「'」や引用符「"」で囲まれた領域）では，日本語も使用できる．

12) 組み込み手続きと同じプログラム名や変数名を使うこと自体はエラーではないが，そのプログラム中では同一名称の組み込み手続きが使えなくなってしまう．

13) Emacs 系のエディタでは，環境変数が適切に設定されていれば，end と打った後に Tab キーを押すと，プログラム名が補完される．

2行目の implicit none は，**暗黙の変数の型**に関する規約[14] を無効にする宣言である．implicit none を宣言することにより，使用するすべての変数を宣言することが義務づけられるが，後で述べるように，文字の打ち間違いなどのエラーの防止に役立つので，この宣言を省略してはならない．

3行目では，実行文中で使用する整数型の変数 i と wa を宣言している．このように，integer i, wa と宣言すると，i と wa は**基本整数型**という変数となる．基本整数型は，通常4バイトの大きさのメモリ領域を使って表現される整数である[15]．

プログラム中では，整数，実数，複素数，文字，論理式など，対象に応じて型の異なる変数を使い分ける．整数型，実数型，複素数型は合わせて**数値型**とよばれる．Fortran 90/95 で使用する変数の型に関しては，付録1.1 に解説があるので参照されたい．なお，implicit none 宣言は，これらの変数の宣言よりも前に記述する必要がある．

implicit none 宣言と整数型変数の宣言

```
program プログラムの名称
  implicit none     ! 最初に implicit none 宣言を必ず行った後，
  integer i, wa     ! 整数型変数 (i と wa) を宣言する
  ... この後に実行文を書く
end program プログラムの名称
```

■do ループによる総和の計算　次に，リスト1.1 の4行目から8行目までの実行文を見てみよう．このうち以下のプログラム部分は，反復演算により合計を求める基本パターンである．

do ループで合計を求める基本パターン

```
wa = 0          ! 最初に変数 wa の値を 0 とする
do i = 1, 100   ! do ループによる反復演算
   wa = wa + i  ! wa に i を加算する
enddo           ! do ループの終端
```

この演算では，最初に合計を求める変数 wa を 0 とし，反復計算により wa に値を加算していくことで合計を得る．最初の行の演算

```
wa = 0
```

は，「両辺が等しい」という意味ではなく，右辺の 0 を左辺の wa に代入するという演算（**ゼロクリア**）を表す．宣言された直後の変数の値は一般に不定であるので，合計を求めるために値を足し込んでいく変数は，必ずゼロクリアする必要がある．同様に，

14) implicit none 宣言がなければ，変数の先頭がアルファベットの順で i から n までの範囲の文字の場合には整数，それ以外では実数となるという規約．implicit none 宣言すれば，この規約は無視してよい．

15) 1ビット (bit) は計算機が扱うデータの最小単位で，0か1かという2つの状態を表す．1バイト (byte) は8ビットのデータである．


```
wa = wa + i
```

は，右辺の wa と i の和を計算して，それを左辺の wa に代入するという演算を表す．なお，加算を表す + という演算子の代わりに -, *, / という演算子を用いれば，それぞれ減算，乗算，除算を行った結果が左辺の wa に格納される（当然ではあるが，乗除算の場合には，wa に意味のある初期値を入れておく必要がある）．演算式の詳細は，1.3.4 項で述べる．

リスト1.1 の5行目にある do 文中の変数 i は，**制御変数**（あるいは **do 変数**，カウンタ）といわれる．制御変数には必ず整数型変数を用いる[16]．制御変数の**始値**と**終値**を 1 および 100，すなわち do i = 1, 100 とすると，i の値が $1, 2, \ldots, 100$ と順に変化して enddo までの間の演算が繰り返し行われる．この do 文から enddo 文までのプログラム範囲は **do ループ**とよばれる．

リスト1.1 の do ループで行われる反復演算のうち，最初に実行される次の演算

```
wa = wa + i
```

では，i が 1 であり，ゼロクリアにより右辺の wa が 0 となっているので，それらの和である 1 が左辺の wa に代入される．do ループの2回目の演算では，i が 2，右辺の wa が 1 であるので，その和である 3 が左辺の wa に代入される．同様の反復演算が，i が 100 となるまで繰り返し続けられる．do ループ中で変数の値がどのように変化するかを確認するために，後述の write 文を do ループ中に一度書いてみるとよい（ただし，そのままでは出力も 100 回行われるので，終値を 10 などとする）．なお，do ループ内では制御変数の値を変更するような代入演算を行ってはならない．

■変数の初期値と変数への代入の順序　変数は，一般に宣言されただけの状態では値が「不定」であり，代入演算や read 文による読み込みなどが行われて，初めて値が確定する．値が不定な変数には 0 以外の無意味な数値が入っている場合もある．このため，リスト1.1 の例では，wa の初期値を 0 と設定してやらないと，合計は正しく求められないことに注意しよう．なお，コンパイラによっては，**オプション**などの設定により，変数の初期値を自動的に 0 にできる場合があるが，プログラムは計算機やコンパイラが変わっても同様に動作するように書くべきであるので，コンパイラの特殊機能に依存するプログラミングは避けたほうがよい．初期値の設定が必要な変数に対しては，必ずプログラム中で所定の値を代入する記述を行う．

また，プログラムの実行文は，後述する制御文を使って処理の流れを人為的に変更しない限り，上から順に実行されていく．このため，同じ変数に代入演算が何度か行われ

16) 過去の FORTRAN では実数型の制御変数が許されたが，廃止予定事項となっている（実数型は 1.3 節で解説する）．一部のコンパイラでは実数型の制御変数はすでに使えないようである．

る場合には，直前に行われた代入演算の結果が変数に格納されていることに注意しよう．プログラム中の変数の値を確認するには，次に述べる write 文をプログラム中に書き加えて値を出力すればよい．

■write 文による出力　リスト 1.1 のプログラムでは，wa に格納された合計の値を write 文により出力している[17]．write( ) の括弧内には，識別子や制御パラメータなどが記述されるが，通常使用するものは「ファイル番号」と「書式（フォーマット）」である[18]．これらをカンマで区切り，左側にファイル番号，右側に書式を書く．また，write( ) に続いて出力対象となる変数を記述する．複数の変数を出力する場合には，それらをカンマで区切る．以上をまとめると，次のような形式となる．

write 文の基本的な使い方

write(ファイル番号，書式) 出力対象，...，出力対象

ファイル番号は出力先を指定するもので，* と書くと，**標準出力**（処理系で定められた出力先で，通常の PC ではディスプレイ画面）に出力される．また，書式に * と書くと，**並び出力**（list-directed output）となり，write 文の後に並ぶ変数の型に合った書式で出力が行われる．並び出力を行った場合の表示形式，たとえば実数の有効桁の末尾まで続く 0 を表示するか否かなどは，コンパイラにより異なる．なお，書式を指定する場合の出力方法については，付録 1.3 に解説があるので参照されたい．リスト 1.1 の例では，**文字定数** 'wa = ' と変数 wa が write 文の出力対象である．文字定数については，付録 1.1 を参照のこと．

■implicit none 宣言について　implicit none 宣言を行うことで，タイプミスなどにより，宣言していない変数をうっかりプログラム中に使用してしまうというエラーを防ぐことができる．リスト 1.2 は，リスト 1.1 で implicit none 宣言を行わず，do ループ内の代入文の右辺で，wa とすべきところを va と打ち間違えた失敗例である．

リスト 1.2　検出しにくいタイプミスを含むプログラム例

```
program loop_err
  ! implicit none 宣言をしない（この行はコメントである）
  integer i, wa
  wa = 0
  do i = 1, 100
     wa = va + i  ! 右辺の wa を va と打ち間違えてしまった!!
  enddo
  write(*, *) 'wa = ', wa
end program loop_err
```

17）出力に print 文を用いることも可能であるが，write 文は print 文より多くの機能があるので，本書では write 文のみを使う．

18）正確には「ファイル番号」は装置識別子，「書式」は書式識別子であるが，本書ではわかりやすい表現を使う．

実際にリスト 1.2 のプログラムを作成して実行してみると，コンパイルおよび実行の際にエラーは表示されないが，実行結果は誤ったものとなるだろう．このように，リスト 1.2 のプログラムには，「エラーは出ないのに，なぜか結果がおかしい」，という非常に困った**バグ**が含まれているのである[19]．しかし，リスト 1.1 のように implicit none 宣言を行っていれば，コンパイルの段階で，打ち間違えた変数に対してエラーが表示されるので，このようなミスを防ぐことができる．

なお，コンパイラによっては，**オプション**を設定することにより，宣言していない変数をプログラム中で使用した際に，エラーを表示できる場合がある[20]．コンパイラにこのような機能がある場合には，これを有効にしたうえで，さらにプログラム中では必ず implicit none 宣言を行うという 2 重のチェックをかけておくとよい．

プログラミングのポイント 1.1

☑ implicit none 宣言を必ず行うこと．

演習 1.1　リスト 1.2 のプログラムを作成して，コンパイルおよび実行時にどのような結果が得られるかを確かめよ．また，このプログラムに implicit none 宣言を加えて，コンパイル時のエラー出力を確認せよ．

1.2.2　自由形式で見やすいプログラムを書く

ところで，リスト 1.1 のプログラムは，**自由形式**で書かれている．自由形式では，1 行につき 132 桁以内の部分にプログラムを自由に書くことができる[21]．しかし，1 行あたり 132 桁が利用できるといっても，実際には，エディタの画面で見やすいように，適当な範囲内に納まるようにプログラムを書く．また，リスト 1.1 のように，Fortran プログラムでは基本的に 1 行に 1 つの宣言文や実行文を書く[22]．長い文は，**継続行**として次の行に分けて記述し，先行する行の末尾に記号 & を付ける．

```
write(*, *) &    ! 先行する行の末尾に & を付ける
  'wa = ', wa    ! 継続行
```

19）プログラム中の誤りをバグ（bug：虫）という．バグを取り除くことをデバッグ（debug）という．

20）gfortran の場合は，gfortran -fimplicit-none loop.f90 のようにオプションを付けてコンパイルする．

21）参考までに，固定形式は，注釈行の印は行の先頭，継続行の印は先頭から 6 文字目に書かねばならない，といった規則に従う記法である．Fortran 95 の自由形式では，より多くの桁数を利用できる．

22）セミコロン「;」で区切ることにより，同一行に複数の実行文を書くことも可能であるが，プログラムが見にくくなるため，この記法は用いないほうがよい．

また，プログラムを書く場合には，do 文と enddo 文に挟まれた範囲などが識別しやすいように，必ず行の先頭を適切に字下げ（インデント）して書く[23]．字下げが行われていないプログラムは読みにくいばかりでなく，エラーを発生させる原因にもなる．

Fortran 90/95 では，従来の FORTRAN[24] と同様に，大文字，小文字の区別はない（これを case-insensitive という）．このため，たとえば write 文は，Write，あるいは WRITE と書いても同様に動作する．また，たとえば変数 num と Num，NUM は同じものなので，これらを異なる変数として使い分けることはできない．昔の記法では，プログラムは大文字を使って書かれることが多かったが，最近はプログラムを小文字で書くことが多い．本書でも基本的に小文字を使って記述する．なお，定数や特殊な変数を大文字で表現するなど，エラーを防ぎ，プログラムをわかりやすくするための工夫は，適宜行うとよいだろう．また，文字列の内容に関しては，大文字・小文字は区別される（付録 1.3 参照）．

プログラミングのポイント 1.2

☑ プログラムは必ず字下げ（インデント）を適切に行い，見やすく書くこと．

1.2.3　次のステップに移るための準備

本節の冒頭で述べたように，これまでのプログラムは，あるディレクトリ（たとえば loop）内で作成されているだろう．新しい問題に対するプログラムを作成する場合には，1 つ上のディレクトリに移動し，そこでまた新しいディレクトリを作成して，その中で作業を行うこととしよう．必要に応じて，以前のディレクトリからプログラムや関連するファイルをコピーして，それらを編集すればよい．

また，正常に動作することが確認されたプログラムに改良を加えていく場合には，元のプログラムを残しておいて，それをコピーしたものを編集するとよい．このようにすると，編集作業に失敗して修復不能となった場合でも，再び元のプログラムから作業をやり直すことができる．

プログラミングのポイント 1.3

☑ 取り扱うテーマごとに個別のディレクトリを作成して，その中にプログラムや関連するデータファイルなど一式を作成する．

☑ プログラムの編集作業を開始する前にバックアップをとる．

23）行頭でスペースキーを何度も押すのは面倒なので，エディタの字下げ機能を利用しよう．前述のように，Emacs 系のエディタでは，Tab キーにより字下げやプログラム名の補完が行える．

24）国際規格（ISO/IEC 1539-1:1991）から，FORTRAN でなく Fortran と表現されるようになった[2]．

演習 1.2　自然数の累乗の和を表す公式に以下のものがある．

$$\sum_{k=1}^{n} k = \frac{1}{2}n(n+1),\quad \sum_{k=1}^{n} k^2 = \frac{1}{6}n(n+1)(2n+1),\quad \sum_{k=1}^{n} k^3 = \frac{1}{4}n^2(n+1)^2$$

n に 100 以下の適当な自然数を設定し，各式の左辺が表す和を do ループを用いて計算した結果と，右辺の値の両方を出力し，それらが一致することを確かめよ[25]．Fortran では，べき乗を求める演算子は ** であり，たとえば k^3 は k ** 3 と表される．

演習 1.3　$a_n = a_{n-1} + a_{n-2}$ $(n \geq 3)$ という関係で定められる整数列を考える（フィボナッチ数列）．$a_1 = 1$，$a_2 = 2$ のとき，この数列の a_1 から a_{10} までの値を出力するプログラムを do ループを用いて作成せよ．

1.2.4　if 文と stop 文

リスト 1.1 に示した合計を求めるプログラムに若干の変更を加えて，条件分岐や演算の停止といった機能を追加しよう．リスト 1.3 は，1 から 100 までの整数のうち，偶数と奇数の和をそれぞれ計算するプログラムである．

リスト 1.3　偶数と奇数の和を反復計算により求めるプログラム

```
program loop_odd_even
  implicit none
  integer i, wa0, wa1
  wa0 = 0                              ! 初期値を 0 とする
  wa1 = 0                              ! 初期値を 0 とする
  do i = 1, 100
     if (mod(i, 2) == 0) then
        wa0 = wa0 + i                  ! 偶数を加算
     else if (mod(i, 2) == 1) then
        wa1 = wa1 + i                  ! 奇数を加算
     else
        stop 'something is wrong !!' ! 異常動作を検出するための stop 文（省略可）
     endif
  enddo
  write(*, *) 'wa0, wa1, wa = ', wa0, wa1, wa0 + wa1
end program loop_odd_even
```

リスト 1.3 の do ループの内部では，制御変数 i が偶数か奇数かを判別するため，組み込み関数の 1 つである余り関数 mod(a, p) を if 文に利用して，条件判定を行っている[26]．if 文の基本的な記述形式は次のように表される．

if 文の基本的な使い方

```
if (論理式 a) then
   ... 実行文 A（論理式 a が真であれば実行文 A が実行される）
```

25）n に過大な値を用いると，総和の値が基本整数型で扱える範囲を超えてしまい，正しい和が得られなくなる場合があるので注意する（付録 1.1 参照）．

26）組み込み関数とは，Fortran 90/95 であらかじめ用意されている関数である（付録 2 参照）．

```
else if (論理式b) then
   ...実行文B (論理式aが真でなく，論理式bが真であれば実行文Bが実行される)
else
   ...実行文C (論理式a，bが真でないときには実行文Cが実行される)
endif
```

上記の実行文Aから実行文Cの箇所には，それぞれ複数の実行文を記述してもよい．if文の括弧内の**論理式**は，単一の論理型変数，あるいは関係演算子や論理演算子を使って書かれた真か偽の値をとる式である（詳細は付録1.2参照）．if文で最も多く用いられるのは，数値を比較する論理式であろう．これは，次のような**関係演算子**を用いて書かれる．

関係演算子を用いる数値の比較

```
if (a >  b)   aがbより大きければ真
if (a >= b)   aがbより大きいか等しいとき真
if (a <  b)   aがbより小さければ真
if (a <= b)   aがbより小さいか等しいとき真
if (a == b)   aがbと等しければ真
if (a /= b)   aがbと等しくなければ真
```

なお，上記以外のif文の使い方については，付録1.2を参照されたい．

リスト1.3のif文で用いられているmod(a, p)は，aとpに整数を用いると，aをpで除したときの余りを整数で返す関数である．これを利用して，mod(i, 2)が0であれば偶数，1であれば奇数と判定し，それぞれの合計wa0，wa1に加算している．プログラムに問題がなければ，mod(i, 2)の値は0か1しかありえないが，リスト1.3では，プログラムの作り間違いなどによる異常な動作を検出するために，

```
else
   stop 'something is wrong !!'
```

という部分を加えている．stop文は，プログラムの処理を停止するために用いられる．実際には，このstop文が実行されることはないはずだが，エラーや予期せぬ動作を防ぐために，最初にプログラムを書くときにはこのような「安全装置」を設けておくとよい．正常な動作が確認されたら，この部分は削除してもよい．

なお，stop文には，この例のように，アポストロフィ「'」で囲んだ文字列を追加することができる（引用符「"」を使用してもよい）．stop文が実行されるときには，その文字列を出力してから処理が終了するので，停止する理由などをここに書いておくと便利である．

1.2.5　無限ループ

今度は，リスト1.1の合計を計算する範囲を，キーボード入力（**標準入力**）で定めるように書き換えてみよう．入力と結果の表示が繰り返し行われるように，**制御変数なしのdoループ**を使ってリスト1.4のように記述する．

リスト1.4　1から入力値までの和を計算するプログラム（無限ループを使用）

```
program loop_inf
  implicit none
  integer wa, n, i
  do                                  ! 制御変数なしの無限ループ(外側のdoループ)
     write(*, *) ' input n (if n <= 0, stop) : '  ! 数値入力を促す表示
     read (*, *) n                    ! 数値を標準入力から読み込む
     if (n <= 0) stop ' good bye ... ' ! 入力値が0以下なら停止
     wa = 0
     do i = 1, n                      ! 総和を計算するdoループ(内側のdoループ)
        wa = wa + i
     enddo
     write(*, *) 'wa = ', wa
  enddo
end program loop_inf
```

このプログラムでは，2つのdoループが使われていて，一方がほかのループに含まれる形になっている．これを**2重ループ**という．外側のdoループは，制御変数なしのdoループとなっているので，do–enddoの間にループから抜け出す仕組みを作っておかないと，反復演算が無限に繰り返されることになる[27]．リスト1.4では，入力された数値が0以下であれば，stop文が実行されて終了する．

このプログラムをコンパイルして実行すると，5行目のwrite文により，次のように数値の入力を促す表示がディスプレイ上に表示される．

```
 input n (if n <= 0, stop) :
100  <- 数値をキーボードから入力した例
```

入力を促す表示の後で**改行**を行いたくなければ，5行目のwrite文を次のように書く．

```
write(*, '(a)', advance = 'no') ' input n (if n <= 0, stop) : '
```

このwrite文の書式については付録1.3で解説されている．

リスト1.4では，このwrite文に続き，入力値を読み込むread文が実行される．read文の基本的な使い方を以下に示す．

read文の基本的な使い方

read(ファイル番号，書式) 入力対象，...，入力対象

27) プログラムを強制的に終了するときには，Ctrl（コントロール）キーを押しながら，cのキーを押す．

read( ) の括弧内には，write( ) の括弧内と同様にして，**ファイル番号**と**書式**をカンマで区切ってこの順序で書く．read 文では，ファイル番号は入力元を指定し，これを * とすると，標準入力（処理系で定められた入力元で，通常の PC ではキーボード）からの入力となる．また，read 文の書式を * とすると，**並び入力**（list-directed input）となり，read 文の後に並ぶ変数の型に合った書式で読み込みが行われる[28]．

リスト 1.4 では，整数型変数 n と同じ型のデータとして読み込みを行う．このため，数字以外の英字や記号をキーボードから入力すると，エラーを起こす．また，3.14 などの実数を入力すると，小数点以下が切り捨てられた整数とみなされて，3 が n に代入される．もし，0.99 と入力すると，切り捨てにより n は 0 となり，処理は終了する．

1.2.6　do ループの始値・終値とストライド（増分値）

do ループの制御変数の始値と終値をそれぞれ m，n とすると，反復演算を行うプログラムは以下のように書かれることが示された．

```
do i = m, n
   ...
enddo
```

リスト 1.1 およびリスト 1.4 では，いずれも制御変数の始値と終値の大小関係は $m \leq n$ となっていて，反復計算における制御変数の増分値は 1 であった．do ループでは，制御変数の終値の後にカンマを付け，その後に整数あるいは整数型変数を書くと，**ストライド**（制御変数の増分値）を指定することができる．ストライドが省略されたときは，ストライドは 1 と解釈される．ストライドを 0 とすることはできない．

ストライド（増分値）が指定された do ループ

```
do i = m, n, k  ! 始値 m，終値 n，ストライド k の do ループ (m <= n のとき)
  wa = wa + i   ! m から n 以下の範囲で，i を k ずつ増加させた演算が行われる
enddo
```

一方，制御変数の始値と終値の大小関係を $m > n$ としてもエラーではなく，この場合には do–enddo の間の演算は 1 度も実行されずに，enddo 文の次の実行文へ処理が移ることになる．制御変数の始値 m と終値 n の関係が $m > n$ である場合には，ストライドに負の値を指定すれば，m から n へ向かって制御変数が減少していく反復演算を行うことができる．以下に，いくつかの例を示す．

```
do i = 1, 4, 2
   write(*, *) i      ! 1 と 3 が順に出力される
enddo

do i = 4, 1, -2
   write(*, *) i      ! 4 と 2 が順に出力される
enddo

do i = 0, 0, -1
   write(*, *) i      ! 0 が 1 つ出力される
enddo
```

28）書式を指定する場合の入力方法については，付録 1.3 に簡単な説明がある．

演習 1.4　上記のように，いろいろな条件の do ループを書いて出力を確認せよ．また，do ループ終了後（enddo 文の直後）における制御変数の値を確認せよ．

1.2.7　exit 文，cycle 文，goto 文

do ループによる反復計算を行う場合に，ある条件が満たされたときには，反復演算を終了してループから抜け出し，enddo 文より後にある処理に移りたい場合や，ループ内の残りの演算を行わずに最初の do 文に戻って反復演算を続けたい場合がある．このような場合には，exit 文や goto 文，cycle 文を使う．次に示すリスト 1.5 のプログラムでは，負の値が入力されたときには総和を計算せず，do 文へ戻り，0 が入力されたら無限ループから抜けて，enddo 以下の行が実行される．

リスト 1.5　1 から入力値までの和を計算するプログラム（exit 文，cycle 文を使用）

```
program loop_exit
  implicit none
  integer wa, n, i
  do
     write(*, *) ' input n (input 0 to stop) : ' ! 数値入力を促す表示
     read (*, *) n      ! n の値を読み取る
     if (n == 0) then
        exit            ! ループから抜けて，enddo 文の次の実行文に移る
     else if (n < 0) then
        write(*, *) ' sorry, input positive n ... '
        cycle           ! これ以降の処理を行わずに，上にある do 文へ戻る
     endif
     wa = 0
     do i = 1, n
        wa = wa + i
     enddo
     write(*, *) 'wa = ', wa
  enddo
  write(*, *) 'exit from do-loop ... '
end program loop_exit
```

この例のように，exit 文が実行されると do ループの演算は終了し，処理は enddo 文の次の実行文に移る．また，cycle 文が実行されると，それ以降の enddo 文までの行は実行されずに，do 文へ戻る．exit 文と cycle 文の基本的なはたらきを以下に示す．

exit 文と cycle 文

```
do i = m, n            ! do 文
   ...
   if (a == b) cycle   ! a と b が等しければ do 文に戻る
   ...
   if (c == d) exit    ! c と d が等しければ do ループから抜ける
   ...
enddo                  ! enddo 文
...                    ! enddo 文の次の実行文
```

リスト 1.5 で使用した exit 文と cycle 文は，いずれも goto 文を使用して記述することができる．また，if 文を用いれば，cycle 文を使わずに済む場合もある．その例をリスト 1.6 に示す．

リスト 1.6　1 から入力値までの和を計算するプログラム（if 文と goto 文を使用）

```
program loop_goto
  implicit none
  integer wa, n, i
  do
     write(*, *) ' input n (input 0 to stop) : '
     read (*, *) n
     if (n == 0) then
        goto 1          ! ループの外にある文番号 1 の continue 文へジャンプする
     else if (n < 0) then
        write(*, *) ' sorry, input positive n ... '
     else
        wa = 0
        do i = 1, n
           wa = wa + i
        enddo
        write(*, *) 'wa = ', wa
     endif
  enddo
1 continue              ! 1 という文番号が付いた continue 文
  write(*, *) 'exit from do-loop ... '
end program loop_goto
```

このプログラムでは，無限ループの enddo 文の次の行に continue 文がある．continue 文は何もしない実行文である．リスト 1.6 の continue 文の先頭には，1 という文番号が付けられており，8 行目の goto 1 が実行されると，演算処理はこの文番号の行へ移る[29]．なお，上記の例では，continue 文を使わずに，下から 2 行目の write 文の先頭に 1 という文番号を付けてもよい．goto 文の基本的なはたらきを以下に示す．

29)　continue 文は，この例のように文番号とともに利用され，栞のように使われることが多い．

goto 文の基本的な使い方

```
       ...
       goto 文番号      ! 処理が goto 文に達すると，
       ...
文番号 実行文           ! 指定された文番号へジャンプする
```

goto 文は，処理の流れがわかりにくくなるため，あまり使用しないほうがよいといわれる．しかし，**多重ループ**から抜けるときには，便利なので利用してもよい．次の例では，**3 重ループ**の中に exit 文が含まれている．

```
do i = 1, 2
   do j = 1, 2
      do k = 1, 2
         write(*, *) 'i, j, k = ', i, j, k
         if (k >= 1) exit
      enddo
   enddo
enddo
write(*, *) 'out of the loop'
```

処理が exit 文に到達すると，1 つ外側の do ループに移動する．しかし，3 重ループの外側への移動ではないので，これを実行すると次のような結果が表示される．

```
i, j, k =          1          1          1
i, j, k =          1          2          1
i, j, k =          2          1          1
i, j, k =          2          2          1
```

このような多重ループの外側へ一気に抜けるには，goto 文を使えば

```
  do i = 1, 2
     do j = 1, 2
        do k = 1, 2
           write(*, *) 'i, j, k = ', i, j, k
           if (k >= 1) goto 1
        enddo
     enddo
  enddo
1 write(*, *) 'out of the loop'
```

と簡単に書くことができる．

演習 1.5　リスト 1.4 のプログラムを改良して，入力値を 2 つの整数 m，n とし（$m \leq n$），その範囲の整数の和 $\sum_{i=m}^{n} i$ を計算するプログラムとせよ．

演習 1.6　演習 1.5 のプログラムを改良して，$m > n$ の場合でもその範囲の整数の和を計算できるようなプログラムとせよ．

演習 1.7　相異なる n 個のものから r 個をとる**順列**と**組み合わせ**の数（${}_nP_r$ と ${}_nC_r$）は，それぞれ ${}_nP_r = n!/(n-r)!$，${}_nC_r = n!/[r!\,(n-r)!]$ と表される．n と r をキーボードから入力して，${}_nP_r$ と ${}_nC_r$ を出力するプログラムを do ループを用いて作成せよ．ただし，$0 \leq r \leq n$ とし，

n は 10 以下の自然数とする．

演習 1.8　100 万以下の自然数 n が入力されたとき，それが**素数**であるかどうかを判定するプログラムを作成せよ．素数とは，「1 あるいは自分自身でしか割り切れない 1 より大きな整数」である．ここでは簡単に，組み込み関数 mod を利用して，除算の余りを調べることにより，素数か否かを判定するプログラムとせよ[30]．

演習 1.9　1 万以下の 2 つの自然数 m，n が入力されたとき，それらの**最大公約数**を求めるプログラムを作成せよ．

演習 1.10　1 万以下の整数が入力されたとき，これを素数の積として表示（**素因数分解**）するプログラムを作成せよ（一例として，除算の余りが 0 となる素数を表示し，その商に対して除算の余りが 0 となる素数を見つける演算を繰り返す，といった方法などが考えられる）．

1.3　実数型変数を用いる反復計算

1.3.1　数値積分を行うプログラム

次に，実数型の変数を用いるプログラムを作成しよう．関数 $f(x)$ の定積分を数値計算により近似的に求めることを**数値積分**という[31]．**台形公式**とよばれる数値積分法では，図 1.1 に示すように，積分区間を n 等分して点 $x_0, x_1, \ldots, x_n$ を定め，$y_k = f(x_k)$ 上の点を直線で結んで作られる台形の面積を合計することにより，定積分の近似値を求める（$k = 0, 1, \ldots, n$）．

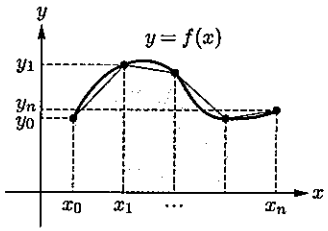

図 1.1　台形公式による数値積分

台形の面積を合計した値 s を求める計算は，次のように表される．

$$s = \frac{1}{2}\Delta x \sum_{k=0}^{n-1}(y_k + y_{k+1}) = \left[\frac{1}{2}(y_0 + y_n) + y_1 + y_2 + \cdots + y_{n-1}\right]\Delta x \quad (1.1)$$

したがって，s を求めるには，積分区間の両端の点の y を 1/2 倍した値とほかの点の y の和を求め，その結果に Δx を乗ずればよい．

台形公式による数値積分を行うプログラムを作成するために，次の例題を考える．

30）簡単には，2 以上 $\sqrt{n}$ 以下の整数で割り切れるかどうかを調べる．平方根を求める組み込み関数 sqrt には整数を用いることができないので（付録 2 参照），$\sqrt{n}$ 以下の最大の整数は，int(sqrt(dble(n))) とする．実際の素数判定プログラムには，より複雑なアルゴリズムが用いられている[3]．

31）数値積分法には，ここで扱う台形公式のほかに多くの手法がある．詳細は文献 [4] などを参照．

[例題 1.1]　台形公式を用いて，次の定積分の近似値を求めるプログラムを作成せよ．

$$S = 6\int_0^1 x(1-x)\,dx \quad (1.2)$$

この例題に対するプログラム例は，リスト 1.7 のようになる．例題 1.1 の定積分の値は 1 であり，積分区間の分割数 n を大きくしていくと，計算された近似値は 1 に近づくことが確かめられる．

リスト 1.7　台形公式により数値積分を行うプログラム

```
program daikei_sekibun
  implicit none
  real(8) dx, x, y, s              ! 倍精度実数型の変数を宣言
  integer i, n                     ! 整数型の変数を宣言
  write(*, *) 'input n : '         ! 入力を促す表示
  read(*, *) n                     ! 分割数 n を読み込む
  if (n < 1) stop 'stop, n < 1'    ! n < 1 なら停止する
  dx = 1.0d0 / dble(n)             ! 分割幅 (台形の高さ)
  s = 0.0d0                        ! 面積をゼロクリア
  do i = 0, n
     x = dx * dble(i)              ! 分割された x 軸上の点の値
     y = x * (1.0d0 - x)           ! x に対応する y の値
     if (i == 0 .or. i == n) then
        s = s + 0.5d0 * y          ! 両端は 1/2 倍して加算
     else
        s = s + y                  ! ほかの点はそのまま加算
     endif
  enddo
  s = 6.0d0 * s * dx               ! 定積分の近似値を計算
  write(*, *) 's = ', s
end program daikei_sekibun
```

リスト 1.7 の 3 行目では，real(8) dx，x，y，s として，**倍精度実数型**の変数 dx，x，y，s を宣言している．倍精度実数型変数は，8 バイトのメモリ領域により表現される実数を表す変数である．この (8) を付けずに，単に real とすると，**基本実数型**となる．基本実数型は，4 バイトのメモリ領域により表される実数型で，**単精度実数型**ともいわれる．単精度および倍精度実数は，10 進数で表したときに，それぞれ約 7 桁および約 15 桁の精度をもつ．技術計算では，単精度実数型変数を用いると計算精度が不足し，結果が不正確な値となることが多い．このため，とくに理由のない限り，実数型変数はすべて倍精度実数型変数として宣言するのがよい（付録 1.5 参照）．なお，単精度実数型と倍精度実数型は，正確にはそれぞれ種別パラメタ（kind）が 4 および 8 の実数型である．実数の型や精度の詳細は，付録 1.1 を参照されたい．

倍精度実数型変数の宣言

```
real(8) x, y, z   ! 倍精度実数型変数 x, y, z の宣言
```

リスト 1.7 の演算の主要部分は，式 (1.1) の計算であり，リスト 1.1 の総和を求めるプログラムと同様に，s をゼロクリアしてから do ループで所定の値を加算していく方法が用いられている．リスト 1.7 のプログラムでは，次の点に注意してほしい．

- 倍精度実数型変数を用いる実行文中では，数値の末尾に d0 が付けられている．
- dble という組み込み関数が用いられている．

数値の末尾に付けられた d0（0 は数字のゼロ）は，数値の精度を倍精度として変数に代入する指示であり，dble() という組み込み関数は，括弧内に書かれた変数（引数）[32] を倍精度実数型変数に変換するものである．これらの詳細を次項以降で解説する．

1.3.2　倍精度実数型変数への数値の代入

リスト 1.7 では，たとえば s = 0.0d0 のように，右辺の数値の末尾に d0 が付けられている．これは数値の精度を倍精度として変数に代入するための指示で，d0 を付け忘れてはならない[33]．たとえば，次のようなプログラム部分を考える．

```
real(8) a, b    ! a, b は倍精度実数型変数
a = 1.2         ! d0 が付いていない !!
b = 3.4         ! d0 が付いていない !!
write(*, *) 'a,  b = ', a,  b
write(*, *) 'a * b = ', a * b
```

このように，右辺の数値に d0 を付け忘れると，詳細は処理系にも依存するが，たとえば次のように出力されてしまう．

```
a,  b =    1.20000004768372        3.40000009536743
a * b =    4.08000027656556
```

しかし，右辺の数値に d0 を付ければ，次のように出力される[34]．

```
a,  b =    1.20000000000000        3.40000000000000
a * b =    4.08000000000000
```

このように，変数の型が倍精度であっても，数値に d0 を付け忘れると精度が低下してしまうことがある．なお，d0 は 10^0 を意味しており，3.14d0（$= 3.14 \times 10^0$）は 0.314d1（$= 0.314 \times 10^1$）あるいは 31.4d-1（$= 31.4 \times 10^{-1}$）などと表記してもよい．

32)　3.2.4 項で述べるように，関数やサブルーチンの括弧内に記述される変数を引数（ひきすう）という．
33)　d0 の代わりに e0 とすると，基本実数型（単精度実数型）となる．数値の精度を設定する一般的な方法については，付録 1.1 を参照．
34)　コンパイラによっては，出力の際に末尾に付く 0 の表示が省略される場合もある．

プログラミングのポイント 1.4

- ☑ 実数を表す変数は，基本的には倍精度実数型変数として宣言する．
- ☑ 倍精度実数型変数に数値を代入するときには，数値の末尾に d0 などの表示を付けること．

1.3.3　型の異なる変数を用いる演算

整数型と倍精度実数型のように，型の異なる変数を同一式中に用いる演算（**混合演算**）は可能であろうか．次のプログラム部分では，組み込み関数により円周率 π の値を求めている．

```
integer n                      ! 整数型変数
real    r                      ! 単精度実数型変数
real(8) pi1, pi2, pi3, pi4     ! 倍精度実数型変数
n = 2
r = 0.0
pi1 = 2.0     * acos(0.0)      ! 右辺は単精度実数 * 単精度実数
pi2 = 2.0d0   * acos(0.0d0)    ! 右辺は倍精度実数 * 倍精度実数
pi3 = dble(n) * acos(dble(r))  ! 右辺は倍精度実数 * 倍精度実数
pi4 = dble(n) * acos(r)        ! 右辺は倍精度実数 * 単精度実数
```

すでに述べたように，dble(n) は引数 n を倍精度実数型に変換する組み込み関数であり，acos(x) は $\cos^{-1} x$ を求める組み込み関数である[35]．上記を実行して，pi1 から pi4 までを順に出力すると，一例として以下の数値が得られる．

```
3.14159274101257      ! 実際の pi は 3.1415926535 8979323846... である
3.14159265358979
3.14159265358979
3.14159274101257
```

この 2 行目と 3 行目の出力に見られるように，右辺の型が倍精度に統一されている場合には，倍精度実数型の有効桁までの精度を有する数値が得られる．一方，1 行目と 4 行目では単精度実数が混在しているため，精度の低い値となっている．このように計算精度が低下することがあるので，倍精度実数型変数への代入演算では，右辺の型を倍精度に揃えておくのが安全である．

上記のような混合演算では，型に優先順位が設けられており，優先される型に合わせて演算がなされる．しかし，実際にプログラムを書く場合には，このような優先順位に依存した書き方よりも，型を揃えて記述するほうが簡単かつ明瞭である．ただし，実数

35)　acos(x) は「総称名」という機能により，単精度および倍精度実数型変数を引数にできる（5.3 節参照）．acos(x) は $0 \leq$ acos(x) $\leq \pi$ の範囲の値を返すので，プログラム中では 2.0d0 * acos(0.0d0) または acos(-1.0d0) とすることにより π の値を簡単に取得できる．なお，$\sin^{-1} x$ と $\tan^{-1} x$ を求める関数 asin(x) と atan(x) は，ともに $-\pi/2 \leq$ asin(x), atan(x) $\leq \pi/2$ の範囲の値を返す．

あるいは複素数のべき乗を求めるときには，2.0d0 ** 3 のように，べき指数に整数を用いる[36]．

なお，型の変換に関係する組み込み関数としては，以下のようなものがある．

```
real(8) d
d = 2.0d0 * acos(0.0d0)
write(*, *) dble(2)     ! 倍精度実数化
write(*, *) d
write(*, *) real(d)     ! 単精度実数化
write(*, *) int (2.5d0) ! 小数点以下を切り捨てて整数化
write(*, *) nint(2.5d0) ! 小数点以下を四捨五入して整数化
```

各関数の意味は右側に書かれたコメントのとおりで，出力結果は以下のようになる．

```
2.00000000000000
3.14159265358979
3.141593
        2
        3
```

1.3.4　演算式の記述に関するその他の注意事項

■加減乗除とべき乗　演算式を記述する際に注意すべき点をあげておく．これまでに見たように，加減乗除は +，-，*，/ という記号で表され，べき乗は ** を用いて計算される．これらは**組み込み演算子**といわれ，この演算子を用いる演算は**組み込み演算**とよばれる．このうち，- は単独の項に付けることが可能であり（単項演算子），以下のように記述することができる．

```
a = - b
```

また，**0 による除算**を行うと演算に支障をきたすので，除算を行う実行文の前には，分母が 0 でないことを確認する if 文を入れる習慣をつけよう[37]．リスト 1.7 では，分割幅を求める演算式で dx = 1.0d0 / dble(n) という除算が行われているが，その前の行で n が 1 以上であることを確認している．

また，初心者がよく間違える記述に，**整数の除算**がある．たとえば，4 の平方根を計算するつもりで次のように書くと，答えは 1 となってしまう．

```
4 ** (1 / 2)
```

36）2.0d0 ** 3 では 2.0d0 を 3 回掛けた値が計算されるが，2.0d0 ** 3.0d0 とすると，処理系によっては計算に対数が用いられて正確な値が得られなかったり，あるいは負の数のべき乗が計算できない場合があるといわれる．このため，べき指数が整数で表せるときは，実数型でなく整数型を用いるほうがよい．

37）0 による除算を行うと，演算が停止するか，処理系によっては NaN（Not a Number，非数）と扱われて計算が継続する．当然，後者の計算結果は意味をなさない．

これは，**整数あるいは整数型変数の除算**の結果は，小数点以下が切り捨てられた整数となるためで，指数部分の (1/2) は 0 となり，4 の 0 乗が計算されて結果は 1 となるのである．したがって，このような演算では実数を用いる必要がある．

■演算順序　次に，演算式中で個々の**演算が行われる順序**を確認しておこう．式中に括弧があれば，その中の演算が先に行われる．括弧が付いていなければ，

- べき乗は，乗除演算より先に行われる．
- 乗除演算は，加減演算より先に行われる．

というルールがある．たとえば，

```
1 + 2 * 3 ** 2
```

という演算は，$1+[2\times(3^2)]=19$ である．また，演算式は左から評価されるので，

```
4 * 6 / 3 * 2
4 * 6 / 3 / 2
```

の 1 行目は $[(4\times6)/3]\times2=16$ であり，2 行目は $[(4\times6)/3]/2=4$ である．なお，べき乗の計算，たとえば

```
2 ** 2 ** 3
```

は $2^{(2^3)}=2^8=256$ となり，右側から評価される．実際に，これらの複雑な演算式を書くときには，括弧を使って演算順序を明示するほうが簡単であり確実である．

■複雑な式の記述方法　最後に，複雑な式の記述方法について補足しておく．たとえば，プログラム中で以下の y を計算する場合を考える（演習 1.24 参照）．

$$y=\frac{(-1)^{n-1}}{(2n-1)^2}e^{-a[(2n-1)\pi/d)]^2t}\sin\frac{(2n-1)\pi x}{d} \tag{1.3}$$

この y の計算式を継続行を使って 1 文で記述してしまうと，演算式が長くなり，たいへん読みづらいプログラムとなるだろう．このような場合には，式の各部分を別の変数で表し，最後にそれらの変数を使って y を記述するとわかりやすくなる．n が整数型，ほかは倍精度実数型の変数とするとき，一例として以下のように記述する[38]．

```
pi = 2.0d0 * acos(0.0d0)
dk = dble(2 * n - 1)
if (dk == 0.0d0 .or. d == 0.0d0) stop 'dk or d = 0'
b  = (-1.0d0) ** (n - 1) / dk ** 2
ei = a * (dk * pi / d) ** 2
ex = exp(-ei * t)
c  = dk * pi * x / d
y  = b * ex * sin(c)
```

38）e^{-x} は，プログラム中では組み込み関数を使って exp(-x) とする．

プログラミングのポイント 1.5

- ☑ 0による除算をしない（除算の前に分母の値を確認する）.
- ☑ 整数の除算に注意する.
- ☑ 長い演算式は部分ごとに分けて記述する.
- ☑ 基本的には混合演算は避け，たとえば倍精度実数型変数への代入文の右辺は倍精度実数型に統一して記述する.
- ☑ べき指数が整数のときには，これを整数（あるいは整数型変数）とする.

演習 1.11　リスト 1.3 で用いた余り関数 mod(a, p) と同じ結果を出力するプログラムを mod を使わずに記述せよ．a, p は整数とする．組み込み関数 int を使ってもよい.

演習 1.12　初項 a が 16.0，公比 r が 0.8 である等比数列 $a_n = ar^{n-1}$ の最初の 10 項の n と a_n を出力するプログラムを作れ．また，この等比数列の第 n 項までの和 S_n は，$S_n = a(1-r^n)/(1-r)$ と表される．do ループを用いて各項の和を求めたときの値と，この公式による S_n の値を比較するプログラムを作成せよ[39].

演習 1.13　次式の標準正規分布（平均 0，分散 1 の正規分布）を $[a, b]$ の範囲で数値積分するプログラムを作成せよ（$a < b$ とする）[40].

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \tag{1.4}$$

演習 1.14　テイラー展開により，e^x は近似的に次のように表される[41].

$$e^x \approx f(x, n) = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} \tag{1.5}$$

do ループを用いて，e の近似値 $f(1, n)$ を $n = 1$ から $n = 10$ まで 10 個求めて出力するプログラムを作れ．また，e の値は組み込み関数を使うと exp(1.0d0) として計算される．この値と，テイラー展開を用いて計算した値の差の絶対値を er とするとき，各 n と er の値を出力せよ．なお，変数 x の絶対値は，組み込み関数を使って abs(x) として求められる.

1.4　反復計算の簡単な応用例

1.4.1　ニュートン法

反復計算を利用する簡単な応用例として，**ニュートン法**（あるいはニュートン－ラフソン法）のプログラムを考える．ニュートン法は，関数 $f(x)$ の導関数 $f'(x)$ を利用して，反復計算により $f(x) = 0$ の解を求める方法である．$f(x) = 0$ の真の解を α とするとき，その近傍にある近似解を x_k とする．テイラー展開の 1 次の項までを用いると，

39）実数を用いる計算では，数学的には同一の式でも，異なる演算により得られた結果は，丸め誤差などが原因で，通常完全には一致しないので注意（付録 1.5 参照）.

40）0 以上の実数 x の平方根は，sqrt(x) として計算される．π の値の設定方法は p. 21 脚注参照.

41）無限級数を有限項で表したときの誤差を，（項の）打ち切り誤差（truncation error）という.

$f(\alpha)$ は次のように近似される.

$$f(\alpha) \approx f(x_k) + f'(x_k)(\alpha - x_k) \tag{1.6}$$

$f(\alpha) = 0$ であるので，これより次の近似解 x_{k+1} が得られる.

$$\alpha \approx x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \tag{1.7}$$

上記の反復計算の過程は，図 1.2 のように表される．$y = f(x)$ という関係を，点 $(x_k, f(x_k))$ を通る接線 $y = f(x_k) + f'(x_k)(x - x_k) \equiv g(x)$ で近似する．この近似式から得られる $g(x) = 0$ の解，すなわち接線が x 軸と交わる点を新しい近似解 x_{k+1} とする．この演算を繰り返して，真の解 α に十分近い近似解を求める．ニュートン法では，$f(x_0) \cdot f''(x_0) > 0$ となるように初期値 x_0 を選ぶと収束解が得られやすいとされている[42].

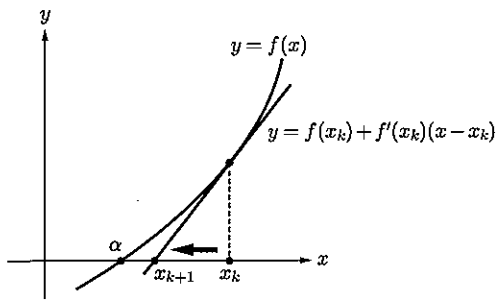

図 1.2　ニュートン法の反復過程

[例題 1.2]　ニュートン法により正の実数 a の平方根 $\sqrt{a}$ を求めるプログラムを作成せよ.

この例題では，$f(x) = x^2 - a$ として，$f(x) = 0$ の正の解をニュートン法により求めればよい．式 (1.7) より，例題 1.2 の近似解を求める関係式は次のように表される.

$$x_{k+1} = x_k - \frac{x_k^2 - a}{2x_k} \tag{1.8}$$

たとえば x_k の初期値を a とすれば，反復計算により近似値は $\sqrt{a}$ に近づく．式 (1.8) を用いて，初期値 a から新しい近似値を繰り返し求めるプログラムを作成すればよい．x_{k+1} と x_k の差が十分小さくなった時点で反復を終了し，近似値や反復回数，誤差などの情報を出力するプログラム例は次のようになる.

42）ニュートン法の収束条件などの詳細は，文献 [4]，[5] を参照.

リスト 1.8　ニュートン法により $\sqrt{a}$ を求めるプログラム

```
program newton_sqrt
  implicit none
  real(8) :: x1, x2, a, er, er0 = 1.0d-6        ! er0 は誤差の許容値
  integer :: k, km = 100                        ! km は最大反復回数
  write(*, *) 'input a : '                      ! 入力を促す表示
  read(*, *) a                                  ! 入力された数値を a に格納
  if (a <= 0.0d0) stop 'a <= 0.0d0'             ! a>0 を対象とする
  x1 = a                                        ! 近似解の初期値を a とする
  do k = 1, km
     x2 = x1 - 0.5d0 * (x1 ** 2 - a) / x1       ! 新しい近似解 x2 を計算
     er = abs(x2 - x1)                          ! x1 と x2 の差の絶対値を er
     if (er < er0) exit                         ! er<er0 なら反復計算終了
     x1 = x2                                    ! 継続するときは x2 を x1 とする
  enddo
  write(*, *) 'kai, k, er = ', x2, k, er        ! 近似解，反復回数，誤差を出力
end program newton_sqrt
```

プログラム中の誤差の許容値 er0 は「**しきい値**」とよばれる．リスト 1.8 の 3 行目では，er0 を宣言する際に，初期値を 1.0d-6（$= 10^{-6}$）と設定している．このように，Fortran 90/95 では，変数の宣言時に初期値を指定することができるが，その宣言文には **2 連コロン**「::」を入れる必要がある．リスト 1.8 の 4 行目でも，同様にして整数型変数 km の宣言と初期値の設定を行っている．なお，このプログラムでは，しきい値を満たす収束解が得られない場合でもプログラムが終了するように，最大反復回数を km とする反復計算を行っている．収束解を求める反復演算では，予想以上に多くの反復回数を要する場合があるので，このように最大反復回数を設定しておくとよい．

プログラミングのポイント 1.6

☑ 宣言文中で初期値を指定する場合には，2 連コロン「::」を入れる．

演習 1.15　k を 2 以上の整数，a を正の実数とするとき，ニュートン法を用いてべき乗根 $\sqrt[k]{a}$ を求めるプログラムを作成せよ．

1.4.2　逐次代入法

方程式 $f(x) = 0$ の解を求める問題において，この方程式が $x = F(x)$ と変形できるとする．このとき，

$$x_{k+1} = F(x_k) \tag{1.9}$$

という形の反復演算を行い，その収束解として $f(x) = 0$ の解を得る方法が**逐次代入法**である．式 (1.9) が収束するためには，方程式 $x = F(x)$ の真の解を α とするとき，$\alpha < x_k$ であるときには，$F(x)$ が $[\alpha, x_k]$ で連続かつ (α, x_k) で微分可能であり，$\alpha < c < x_k$ なる c に対して，$|F'(c)| < 1$ であればよいとされている[43]．

演習 1.16　次式は，微小振幅波の分散関係式といわれる．

$$\frac{L}{T} = \sqrt{\frac{gL}{2\pi} \tanh \frac{2\pi h}{L}} \tag{1.10}$$

ここで，g は重力加速度，L，T，h は順に波の波長，周期，水深を表す．周期 T と水深 h が与えられたとき，式 (1.10) を満足する波長 L を逐次反復法により求めるプログラムを作成せよ．たとえば，h が 2.5 m，T が 5 秒のときに波長は何 m となるか[44]．

1.4.3　モンテカルロ法

モンテカルロ法あるいはモンテカルロ・シミュレーションは，乱数を利用する反復計算により，問題の近似解を求める計算法である．モンテカルロ法を用いて，円周率を求める次の例題を考えよう．

[例題 1.3]　0 以上 1 未満の範囲に一様に分布する疑似的な乱数 x，y を発生させることにより，x-y 平面上で $0 \le x, y < 1$ の範囲 R に含まれる点 (x, y) を定める．図 1.3 に示すように，この点を多数発生させて，そのうち原点を中心とする半径 1 の四分の一円 C に含まれる点の個数をカウントする．N 個の点を発生させたときに，n 個の点が C に含まれる場合，N を大きくしていくと，n/N という比は C と R の面積比，すなわち $\pi/4$ に近づくと考えられる．これを利用して，円周率 π を推定するプログラムを作成せよ．

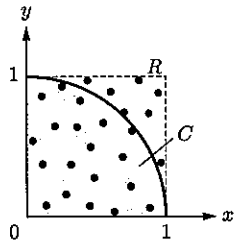

図 1.3　四分の一円と乱数により発生させた点群

ここでは，題意の $[0, 1)$ の範囲に一様に分布する疑似乱数を生成するために，組み込みサブルーチン random_number を利用する．組み込みサブルーチンとは，Fortran 90/95 であらかじめ用意されているサブルーチンである（付録 2 参照）．関数は y = sin(x) のように実行文中に関数名を記述して使うが，サブルーチンは call 文を使って呼び出す（第 3 章参照）．すなわち，call random_number(x) とすると，x に上記の乱数が設定される．プログラム例をリスト 1.9 に示す．

43)　逐次代入法の詳細は，文献 [4] を参照．

44)　たとえば，L の初期値を h とすればよい．$\tanh x$ は組み込み関数を使って tanh(x) として計算できる．水深 2.5 m で周期が 5 秒のとき，波長は約 23 m となる．

リスト 1.9　モンテカルロ法により円周率 π を計算するプログラム例

```
program monte_pi
  implicit none
  real(8) x, y, pi, pi0
  integer :: n, i, im = 2 ** 20 ! im は乱数により定められる点の総数
  pi0 = 2.0d0 * acos(0.0d0)     ! pi0 は組み込み関数により定めた円周率
  n = 0                         ! n をゼロクリアする
  do i = 1, im
     call random_number(x)      ! x に乱数を設定
     call random_number(y)      ! y に乱数を設定
     if (x ** 2 + y ** 2 <= 1.0d0) n = n + 1 ! 条件を満たす点の総数をカウント
  enddo
  pi = 4.0d0 * dble(n) / dble(im)  ! モンテカルロ法により推定された円周率
  write(*, *) 'pi, pi0, er = ', pi, pi0, pi - pi0 ! 結果を出力
end program monte_pi
```

リスト 1.9 のプログラムでは，反復回数を定める整数型変数 im に 2^{20} という大きな値を設定しているが，変数には代入可能な最大値が定められており，それより大きい数を扱うことはできないので注意する（付録 1.1 参照）．プログラムの 4 行目では，im の宣言時に初期値を設定するため，**2 連コロン**「::」を入れている．

1.5　入出力に関する機能

1.5.1　出力リダイレクション

これまでに作成したプログラムでは，実行結果はディスプレイ上に表示された．しかし，多量の実行結果が出力される場合や，計算結果のグラフを描いたりする場合には，実行結果をファイルに書き出したほうが便利である．**出力リダイレクション**という操作を行うと，出力結果をファイルに書き出すことができる．たとえば，リスト 1.1 の実行ファイル a.out を次のようにして実行する[45]．

```
./a.out > output.d
```

すると，ディスプレイには何も出力されないが，同じディレクトリ内に output.d というファイルが作られて，その中に出力結果が書き込まれる．output.d の内容は，プログラムを作成するときに使用したテキストエディタで読むことができる．

上記の出力リダイレクションでは，既存のファイルが存在する場合に，その内容は**上書き**されてしまうが，> の代わりに >> を用いると，既存のファイルの末尾に追加される形で出力される．

45)　UNIX PC ではシェルウィンドウ，Apple PC ではターミナルでこのように入力する．Windows PC では，コマンドプロンプトで a.exe > output.d と入力する．なお，処理系でテキストファイルの拡張子が指定される場合（たとえば .txt に限られる場合）には，それを用いる（output.txt など）．

また，コンパイルの際に，**エラーメッセージ**が多量に画面上に出力される場合には，たとえば次のようにしてコンパイルを行ってみよう[46]．

```
gfortran loop.f90 > & err.d
```

すると，通常の出力結果に加えてエラーメッセージがファイル err.d の中に書き込まれるので，エディタで開いてその内容を読むことができる．このように，エラーメッセージを出力リダイレクションするには，> & を用いる．ところで，エラーを修正する場合には，ある箇所のエラーにより，正しく書かれた後述の部分がエラーと判定されることがあるので，エラーメッセージの先頭に位置するものから順に解決していくのがよい．

プログラミングのポイント 1.7

- ☑ 長いエラーメッセージは，出力リダイレクションにより，ファイルに書き出すと読みやすい．
- ☑ エラーメッセージは先頭に表示されたものから順に解決する．

演習 1.17　リスト 1.1 のプログラムを用いて，出力リダイレクションの機能を確認せよ．また，プログラム中に文法エラーを作り，コンパイル時のエラーメッセージに関する出力リダイレクションの機能を確認せよ（PC でその機能が使える場合）．

1.5.2　入力リダイレクション

出力リダイレクションとは逆の操作として，**入力リダイレクション**がある．入力リダイレクションを行うには，キーボード入力の代わりに，データが記入された入力ファイルを用意しておいて，これをプログラム実行時に読み込ませる．たとえば，リスト 1.4 のプログラムを実行する際には，キーボードから整数値を入力した．今度は，次のような内容のテキストファイルを用意して，これを input.d という名前を付けて保存する．

```
10
100
1000
0
```

リスト 1.4 のプログラムでは無限ループが用いられているので，処理を終了させるために最終行には 0 と書く．また，最終行の数値の末尾でリターンキー（あるいは Enter キー）を押して**改行**しておかないと数値がうまく読み取れない場合があるので，データを作成する際には最終行の数値（上記の例では 0）の後で必ず改行する習慣を付けよう．

次に，リスト 1.4 の実行ファイル a.out を次のようにして実行する．

```
./a.out < input.d
```

46)　この方法は，Windows PC のコマンドプロンプトでは使えない．

すると，キーボードから 10，100，1000 と入力したときと同じ結果がディスプレイに表示された後，プログラムは終了する．なお，上記のように，input.d の最終行にプログラム中の無限ループを終了させるために 0 を記入しておかないと，「input.d から読み込むべきデータがない」という表示が出て異常終了してしまう．

上記の結果を出力リダイレクションにより，output.d へ出力するには，以下のようにすればよい．

```
./a.out < input.d > output.d
```

なお，stop 文の後に付けられた文字列は，通常の出力結果ではなく，エラーメッセージと扱われるので，これも output.d へ出力する場合には，上記の > を > & とする．

1.5.3　open 文と close 文を用いる入出力ファイル操作

■open 文と close 文の使い方　プログラム中に open 文を記述してファイルにアクセスし，ファイル入出力を行うことができる．一般のプログラムでは，複数の入力および出力ファイルが扱われるので，前項のリダイレクション機能よりもこの方法が多く用いられる．次の例題を考えよう．

[例題 1.4]　x，y を $-5 \leq x, y \leq 5$ の範囲でそれぞれ $d = 10/(n-1)$ 刻みで変化させ（n は 3 以上の整数），各 (x, y) に対する $z = \sin x \cos y$ の値を出力するプログラムを作成せよ．open 文を用いて n をファイル入力し，x，y，z をファイル出力せよ．また，出力ファイルの数値を（gnuplot の splot などを利用して）3 次元グラフとして表示せよ．

リスト 1.10 にプログラム例を示す．

リスト 1.10　入出力ファイルを利用するプログラム

```
program file_io
  implicit none
  real(8) d, x, y, z
  integer :: n, i, j, fi = 10, fo = 11 ! 初期値設定する場合には :: を付ける
  open(fi, file = 'input.d')           ! 入力ファイルを開く
  open(fo, file = 'output.d')          ! 出力ファイルを開く
  read(fi, *) n                        ! 入力ファイルから n を読み込む
  close(fi)                            ! 入力ファイルを閉じる
  if (n < 3) stop 'stop, n < 3'        ! n の値をチェック
  d = 10.0d0 / dble(n - 1)             ! x,y の増分 d を設定
  do j = 1, n
     y = -5.0d0 + dble(j - 1) * d      ! y の値を設定
     do i = 1, n
        x = -5.0d0 + dble(i - 1) * d   ! x の値を設定
        z = sin(x) * cos(y)            ! x,y から z を計算
        write(fo, '(3e12.4)') x, y, z  ! 書式を利用してファイル出力
     enddo
     write(fo, *) '' ! gnuplot 描画の際に直線が引かれないように空行を入れる
  enddo
  close(fo)          ! 出力ファイルを閉じる
end program file_io
```


整数値（たとえば 31）が 1 つ記入されたファイル input.d を同じディレクトリ内に用意して，リスト 1.10 のプログラムを実行する．すると，input.d の数値が n に読み込まれて，2 重ループ内で x，y，z が計算され，それらが output.d という出力ファイルに書き出される．図 1.4 のように，open 文を用いることにより，入出力ファイルが指定される．

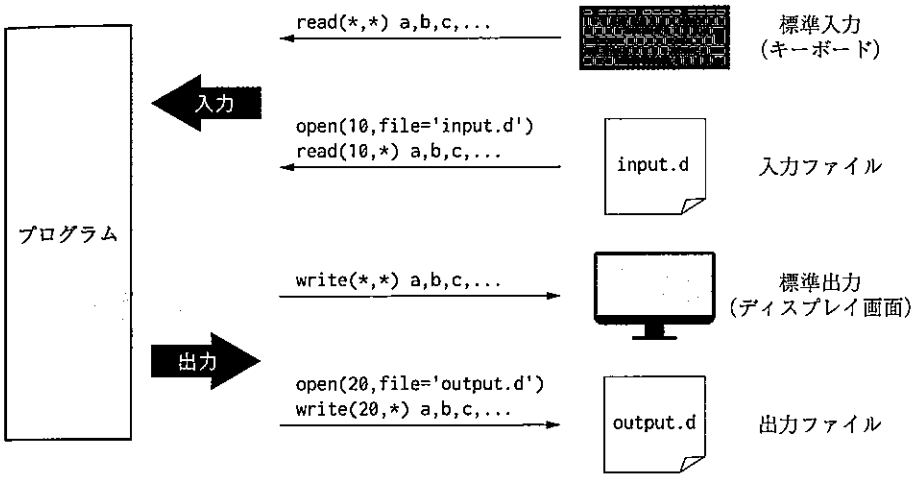

図 1.4　プログラムと入出力

なお，リスト 1.10 では，内側のループの enddo 文の直後で，**空行**（空白の行）を出力している．これは，gnuplot で描画される直線を区切るためのものである（付録 3 参照）．

また，リスト 1.10 では，x，y，z をファイル出力する際に，

```
write(fo, '(3e12.4)') x, y, z
```

として，write 文で出力の**書式**（フォーマット）'(3e12.4)' が指定されている．並び出力により write(fo, *) ... と出力してもよいが，書式を指定すると出力結果が見やすくなり，1 行に多くの出力を行っても途中で改行されることがない[47]．上記の書式

47）ただし，Fortran 90/95 の規格では，1 行に出力する文字数に関する規定がないため，コンパイラによっては 1 行あたりの最大出力文字数が指定されている場合がある（たとえば，1,024 文字までなど）．コンパイラを変えて実行した際に，出力時にエラーが出る場合は，この点を確認してみよう．

3e12.4により，3つの実数が指数形式で小数点以下4桁まで表示される（書式の詳細は，付録1.3を参照）．

リスト1.10のプログラムを実行すると，同じディレクトリ内にoutput.dというファイルがなければそれが新しく作られ，ファイルがすでに存在していれば，そこに演算結果が**上書き**される．また，入力ファイルinput.dはプログラムではないので，その内容を変更しても再コンパイルは不要である．このため，入力パラメータを変化させる計算では，プログラム中に直接数値を打ち込むよりも，入力ファイルを利用するほうが扱いやすい．

入力ファイルからの読み取り，あるいは出力ファイルへの書き出しを行う手順をまとめると，次のように表される[48]．

open文とファイル入出力

1. open文を用いて，入出力ファイル名とファイル番号を結びつける．
   open(入力あるいは出力ファイル番号, file = ' ファイル名')
2. 次に，そのファイル番号をread文あるいはwrite文に用いて入出力を行う．
   read (入力ファイル番号, *) a, b, ...
   write(出力ファイル番号, *) c, d, ...
3. 入出力が終了したら，ファイルを閉じる
   close(入力あるいは出力ファイル番号)

ファイル番号には0以上の整数を用いる．ただし，ファイル番号5と6は，open文で使用されていなければ，それぞれ標準入力（通常キーボード入力）と標準出力（通常ディスプレイへの出力）として使用される．つまり，read(5, *)はread(*, *)と同じであり，write(6, *)はwrite(*, *)と同じである．このため，ファイル番号として5と6は使用しないほうがよい．なお，入力ファイル番号と出力ファイル番号を取り違えないようにしよう．入力ファイルに対してwrite文で出力を行うと，せっかく作成した入力ファイルが壊れてしまう．

入出力ファイルを閉じるには，**close文**を用いる．ファイル入出力の処理が終了したら，誤った処理を防ぐためにも，ただちに

```
close(ファイル番号)
```

としてファイルを閉じるのがよい．とくに，入力ファイルは，読み込みが終了したらすぐにclose文で閉じるのが安全である．

■open文と入出力の制御　open文には，いろいろな指定子を用いて，入出力時の処理を制御することができる．たとえば，入力ファイルを「読み取り専用」とし，誤って書き込むことを防止するには，次のようにaction = 'read'をopen文の括弧内に記述する[49]．

```
open(fi, file = 'input.d', action = 'read')
```

このように記述すると，入力ファイルに対してwrite(fi, *)という書き込みを行おうとしたときにエラーが表示され，処理は停止する．actionに'write'を指定すれば，書き込み専用となる．何も指定しないと，'readwrite'，すなわち読み書きいずれも可能な状態でファイルが開かれる．

また，ファイルオープンの成功あるいは失敗を確認するには，次のようにiostatを記述して，整数型変数（以下の例ではis）にその値を代入する．

```
open(fi, file = 'input.d', iostat = is)
```

ファイルのオープンに成功するとisには0が入り，失敗すると0以外の数値が入る．ここでは，プログラム中の通常の代入文と異なり，右辺の変数isに結果が代入されることに注意しよう．左辺は定められた記述名であり，常にiostatと記述する（大文字でもよい）．たとえば，入力ファイルを読み取り専用で開き，もし入力ファイルが存在しないときには処理を停止させるには，次のようにすればよい．

```
integer :: fi = 10, is
open(fi, file = 'input2.d', action = 'read', iostat = is)
if (is /= 0) stop 'cannot open file'
```

また，ファイルに含まれるデータ数が不明である場合には，以下の例のように，read文にiostat指定子を用いて，データの終端に到達したかどうかを確認できる．

```
integer iost, iv
open(10, file = 'input.d')        ! 入力ファイルを開く
do                                ! 制御変数なしの無限ループ
   read(10, *, iostat = iost) iv  ! iostat指定子の値をiostに代入
   if (iost < 0) exit             ! ファイル終了条件が検出されたらexit
   write(*, *) 'iv = ', iv
enddo
```

上記の例のように，iostat指定子の値は右辺にある整数iostに代入され，ファイル終了条件が検出された場合には，iostの値は負となる．

一方，リスト1.10のように，出力ファイルを次のようにして開くと，すでにファイルoutput.dが存在する場合には，その内容が失われて新しい結果が出力される．

```
open(fo, file = 'output.d')
write(fo, *) ...
```

もし，output.dが存在して，その内容に追加する形式で出力したい場合には，以下のようにposition = 'append'という指定子を加えればよい．

48) open文でファイルを開かずに，直接write(10, *) ...のように書き出すと，通常fort.10というファイルが作られてそこに出力される．簡単にファイル出力を行うには，この方法を用いてもよい．

49) open文の括弧内の記述では，アポストロフィ「'」の代わりに引用符「"」を使うこともできる．

```
open(fo, file = 'output.d', position = 'append')
write(fo, *) ...
```

このようにすると，既存の内容の末尾に新しい出力結果が追加される[50]．

なお，以上の例では入出力ファイル名はプログラム中に直接記述されていたが，文字型変数を用いてファイル名を定めることも可能である（p. 234 参照）．

■gnuplot による描画　さて，例題 1.4 に対するリスト 1.10 のプログラムがうまく実行できたら，次に出力ファイル output.d のデータを gnuplot により 3 次元グラフとして表示してみよう．gnuplot を起動して，gnuplot のプロンプトに次のように入力する．

```
gnuplot> set hidden3d
gnuplot> splot 'output.d' with lines
```

1 行目は隠線処理を行うためのコマンドで，2 行目は出力ファイルの数値を 3 次元表示するためのコマンドである．上記のように，with lines を指定して，数値が表す点を直線で結んで表示する場合には，リスト 1.10 のように，空行を入れてデータを区切る必要がある．描画の例を図 1.5 に示す．

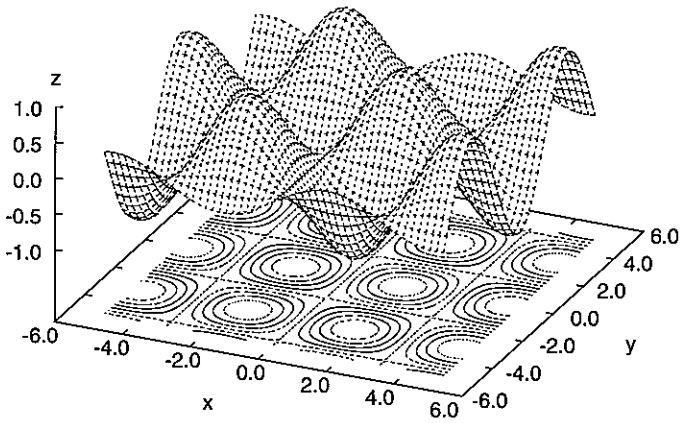

図 1.5　$z = \sin x \cos y$ の 3 次元表示

なお，gnuplot> set contour と入力した後に上記の splot ... を入力すると，図 1.5 のように等高線を x-y 平面上に表示することができる．さらに，gnuplot> set pm3d としてから splot ... を入力すると，面が塗りつぶされた表示が行われる．バージョン 4.0 以上の gnuplot であれば，マウス操作が可能なので試してみよう．

50）上書きされないようにするには，既存の出力ファイルをほかのディレクトリに移動するか，あるいは別の名前のファイルとしておくのが簡単である．

演習 1.18　演習 1.12 で作成した等比級数 a_n の値をディスプレイに表示するプログラムを変更し，n と a_n の値を出力ファイル output.d へ書き出すものとせよ．また，gnuplot を使用して，output.d に出力された n と a_n の関係をグラフに表示せよ[51]．

演習 1.19　$[-1, 1]$ の範囲を $n-1$ 等分して得られる n 個の点を $x_1, x_2, \ldots, x_n$ とする．x_i の値と双曲線関数 $\sinh x_i = (e^{x_i} - e^{-x_i})/2$，$\cosh x_i = (e^{x_i} + e^{-x_i})/2$，$\tanh x_i = \sinh x_i / \cosh x_i$ の値をファイルに出力するプログラムを作成せよ（$i = 1, 2, \ldots, n$）[52]．ファイル出力の際には，リスト 1.10 にあるような書式（たとえば 4e12.4）を用いるとよい．また，gnuplot を用いてグラフ表示してみること[53]．

本章では，do ループを用いる反復計算を中心に，条件に応じて処理を選択する if 文，反復演算の制御を行う exit 文や cycle 文，そしてファイルの入出力操作などを解説した．実際のプログラミングで重要となる配列や副プログラムは後の章で学ぶが，本章の内容を理解しただけでも，演習問題にあるような基本的なプログラムを作ることができるだろう．また，本書では gnuplot による描画をたびたび利用しているが，これは計算結果の理解に役立つばかりでなく，プログラムの動作確認のためにも有効である．プログラミングとともに，描画ツールの使い方にも慣れてほしい．以下に，プログラム作成時の要点をまとめておく．

プログラミングのポイント 1.8

☑ プログラムは一度にすべてを作らないで，動作を確認しながら（コンパイルと実行を繰り返しながら）少しずつ作成していく．

☑ プログラムの動作を確認したり，エラーの原因を探すときには，以下の操作を行ってみよう．

1. stop 文をプログラムの途中に入れて，その位置まで演算が正しく実行できるかどうかを調べる．
2. 疑わしい実行文の先頭に！を付けてコメントとしてみる．
3. write 文を使って変数の値を納得のいくまで書き出す．

演習 1.20　x-y 平面上で，原点を中心とし，半径が r (>0) である円の円周上の点 P は $x = r\cos\theta$，$y = r\sin\theta$ と表される（$0 \le \theta \le 2\pi$）．θ を 0 から 2π まで $\Delta\theta = 2\pi/n$ 刻みで変化させ（n は 3 以上の整数），各 θ に対する円周上の点の x，y 座標をファイルに出力せよ．また，これらの点を結ぶ内接多角形を gnuplot により表示せよ．

演習 1.21　演習 1.20 の多角形の面積と円の面積の差の絶対値 $|e|$ を求め，分割数 n と $|e|$ の値をファイルに出力するプログラムを作成せよ．また，n と $|e|$ の関係を gnuplot を用いてグラフに表示せよ．

51）縦軸を対数表示する場合には，gnuplot> set logscale y と指定してから，gnuplot> plot 'output.d' として描画する．元に戻すには，gnuplot> unset logscale y とする．

52）$\sinh x$ などは，組み込み関数を用いて sinh(x) などとして求めることも可能（付録 2 参照）．

53）ファイル output.d に出力された 1 カラム目のデータと 3 カラム目のデータのグラフを描画するには，gnuplot> plot 'output.d' using 1:3 とする（付録 3 参照）．

演習 1.22　実数 a, b, c ($a \neq 0$) をキーボードから入力すると，**2次方程式** $ax^2+bx+c=0$ の**判別式** $D=b^2-4ac$ の値と，方程式の解を表示するプログラムを作成せよ[54]．解が虚数となる場合は，実部と虚部をそれぞれ表示するものとせよ．

演習 1.23　以下の $K(k)$ を求める積分を**第1種完全楕円積分**といい，べき級数として項別に積分すると，下記のような展開式として表されることが知られている ($0 \leq k^2 < 1$)．

$$K(k) = \int_0^{\pi/2} \frac{1}{\sqrt{1-k^2\sin^2\theta}}\,d\theta = \frac{\pi}{2}\left(1+\frac{1^2}{2^2}k^2+\frac{1^2\cdot 3^2}{2^2\cdot 4^2}k^4+\frac{1^2\cdot 3^2\cdot 5^2}{2^2\cdot 4^2\cdot 6^2}k^6+\cdots\right) \tag{1.11}$$

展開式の右辺の括弧内の第 n 項までを用いて，$K(k)$ を近似的に求めるプログラムを作成せよ[55]．また，第 n 項まで用いた近似値と第 $n-1$ 項までの近似値との差 e を出力し，n が大きくなると $|e|$ が0に近づくことを確認せよ (n は2以上の整数)．

演習 1.24　一端が原点に一致するように x 軸に沿って置かれた長さ L の棒がある．棒の**熱伝導率**は κ とする．この棒の両端の温度を0に保ち，棒の初期温度分布を $T(0,x)=x$ ($0 \leq x \leq L/2$) および $L-x$ ($L/2 \leq x \leq L$) としたとき，時刻 t における**温度分布** $T(t,x)$ は，フーリエ級数を利用すると次式のように表される．

$$T(t,x) = \frac{4L}{\pi^2}\left(e^{-\lambda_1^2 t}\sin\frac{\pi x}{L} - \frac{1}{3^2}e^{-\lambda_3^2 t}\sin\frac{3\pi x}{L} + \frac{1}{5^2}e^{-\lambda_5^2 t}\sin\frac{5\pi x}{L} - \cdots\right) \tag{1.12}$$

ここに，$\lambda_n^2 = \kappa(n\pi/L)^2$ である．上記の括弧内の級数を50項までとり，$\kappa=0.01$，$L=1$ として，$t=0,1,\ldots,10$ のときの温度分布を求めるプログラムを作成せよ（演算式の記述方法に関しては，1.3.4項の式(1.3)を参照せよ）．$0 \leq x \leq L$ の範囲を m 分割したときの各点 x_i ($i=1,2,\ldots,m+1$) における温度 T_i を，上記の時刻ごとにファイル出力すればよい（m は2以上の整数）．また，出力ファイルのデータを gnuplot で表示せよ．級数の項数を増減させると表示はどのように変わるだろうか．

54)　桁落ちの影響を小さくするための方法については，付録1.5参照．

55)　円周率 π を変数に設定する方法は，p.21の脚注を参照．多桁数表によると，一例として $k=0.64$ のとき，$K(k)=1.78423\,63259\,46738\cdots$ である．

第2章 ▶ 配列を用いるプログラミング

2.1　配列の概要

配列 (array) は，固体力学や流体力学などの数値計算プログラムを書くためには必要不可欠である．配列のわかりやすい例は，ベクトルや行列である[1]．ベクトル $\boldsymbol{x}$ の第 i 成分 x_i や，行列 A の i 行 j 列要素 $a_{i,j}$ のように，整数の添字が付く複数のデータをまとめて表現できる変数が配列であると考えればよい．

詳しくは次節以降で解説するが，たとえば図2.1のように，`real(8) x(3)` と宣言すると，倍精度実数を格納できる「枠」が3個用意されるとイメージしてみよう．

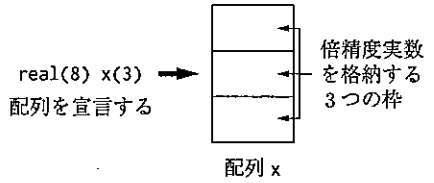

図2.1　1次元配列のイメージ

そして，図2.2に示すように，3次元ベクトル $\boldsymbol{x}$ の成分 (x_1,x_2,x_3) を，プログラム中で3個の枠に順に入れておく．すると，ベクトル $\boldsymbol{x}$ は配列 x，またベクトル成分 x_i は配列要素 `x(i)` と表され，それらを各種の演算で利用できる ($i, \mathrm{i}=1,2,3$)．

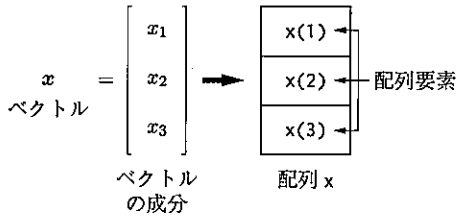

図2.2　1次元配列にベクトルを格納する

たとえば，ベクトル成分 x_i の総和 wa を求めたい場合は，do ループを使って，次のように計算できる．

```
wa = 0.0d0
do i = 1, 3
   wa = wa + x(i)
enddo
```

1)　本書では，ベクトルは，適当な座標系により成分表示されているとする．